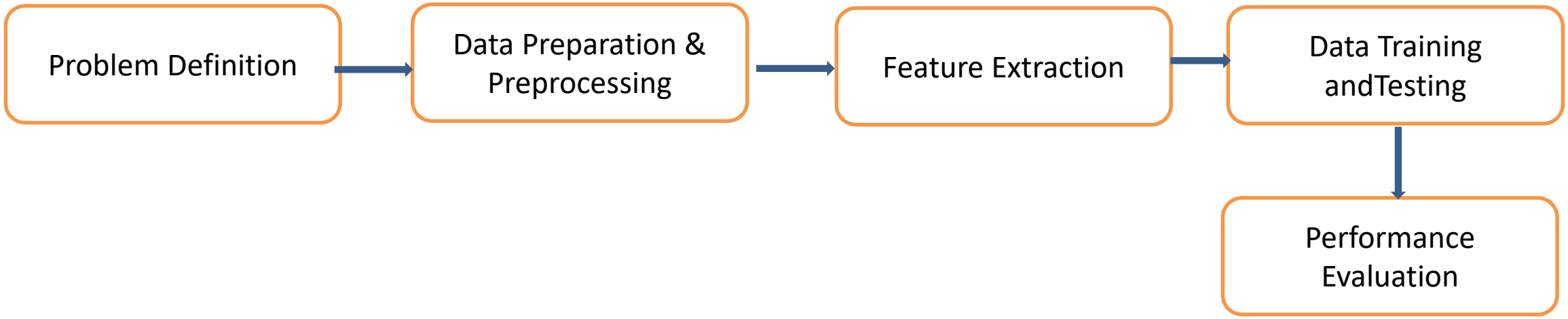
Lecture 3: Data Analysis and Preprocessing
Part 1

Md. Shahriar Hussain
ECE Department, NSU

Steps of solving ML problems



ML Tools

- **Anaconda**: Anaconda is a distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.), that aims to simplify **package management and deployment**

- Anaconda Navigator is a desktop graphical user interface (GUI) included in Anaconda distribution

- It allows users to launch applications and manage conda packages, environments and channels without using command-line commands

ML Tools

- **Jupyter Notebook:** Jupyter Notebook App is a server-client application that allows editing and running notebook documents via a web browser

- **Google Colab**: free cloud service hosted by Google that runs on cloud. It provides free GPU support

- **WEKA**: Weka is a collection of machine learning algorithms for data mining tasks. It contains tools for data preparation, classification, regression, clustering, association rules mining, and visualization.

Data Set Collection

- **Kaggle**: a subsidiary of Google LLC - online community of data scientists and machine learning practitioners.

- Kaggle allows users to find and publish data sets, explore and build models in a web-based data-science environment, work with other data scientists and machine learning engineers, and enter competitions to solve data science challenges

Python Libraries for Data Science

- Popular Python toolboxes/libraries:
  - NumPy
  - SciPy
  - Pandas
  - SciKit-Learn

- Data Visualization libraries
  - matplotlib
  - Seaborn

Python Libraries: **NumPy**

- introduces objects for **multidimensional arrays** and **matrices**, as well as functions that allow to easily perform advanced **mathematical** and **statistical** operations on those objects

- provides **vectorization** of mathematical operations on arrays and matrices which significantly improves the performance

- many other python libraries are built on NumPy

Python Libraries: **SciPy**

- collection of algorithms for **linear algebra, differential equations, numerical integration, optimization, statistics** and more
- part of SciPy Stack
- built on NumPy

Python Libraries: **Pandas**

- adds data structures and tools designed to work with **table-like data** (similar to Series and Data Frames in R)
- provides tools for data manipulation: **reshaping, merging, sorting, slicing, aggregation** etc.
- allows **handling missing data**

Python Libraries: **SciKit-Learn**

- provides machine learning algorithms: **classification, regression, clustering, model validation etc.**

- built on NumPy, SciPy and matplotlib

Python Libraries: **Matplotlib**

- python **2D plotting** library which produces publication quality figures in a variety of hardcopy formats
- a set of functionalities similar to those of MATLAB
- **line plots, scatter plots, barcharts, histograms, pie charts** etc.
- relatively low-level; some effort needed to create advanced visualization

Python Libraries: **Seaborn**

- based on matplotlib
- provides **high level interface** for drawing attractive statistical graphics
- Similar (in style) to the popular ggplot2 library in R

Loading Python Libraries

In [ ]:

```
#Import Python Libraries
import numpy as np
import scipy as sp
import pandas as pd
import matplotlib as mpl
from matplotlib import pyplot as plt
import seaborn as sns
```

Press `Shift+Enter` to execute the *Jupyter* cell

Reading data using pandas

```
In [ ]: #Read csv file
        df = pd.read_csv("http://rcs.bu.edu/examples/python/data_analysis/Salaries.csv")
```

Note: The above command has many optional arguments to fine-tune the data import process.

Exploring data frames

In [3]:
```
#List first 5 records
df.head()
```

Out[3]:

	rank	discipline	phd	service	sex	salary
0	Prof	B	56	49	Male	186960
1	Prof	A	12	6	Male	93000
2	Prof	A	23	20	Male	110515
3	Prof	A	40	31	Male	131205
4	Prof	B	20	18	Male	104800

Data Frame data types

Pandas Type	Native Python Type	Description
object	string	The most general dtype. Will be assigned to your column if column has mixed types (numbers and strings).
int64	int	Numeric characters. 64 refers to the memory allocated to hold this character.
float64	float	Numeric characters with decimals. If a column contains numbers and NaNs(see below), pandas will default to float64, in case your missing value has a decimal.
datetime64, timedelta[ns]	N/A (but see the datetime module in Python's standard library)	Values meant to hold time data. Look into these for time series experiments.

Data Frame data types

```
In [4]: #Check a particular column type
        df['salary'].dtype # creating a list
```

```
Out[4]: dtype('int64')
```

```
In [5]: #Check types for all the columns
        df.dtypes
```

```
Out[4]: rank          object
        discipline    object
        phd           int64
        service       int64
        sex           object
        salary        int64
        dtype: object
```

Data Frames attributes

- Python objects have *attributes* and *methods/functions*.
- df.describe ()
  # methods/function
- df.columns # attributes of the dataset
- Python methods/functions have *parenthesis.*
- Python attributes have NO *parenthesis.*

df.attribute	description
dtypes	list the types of the columns
columns	list the column names
axes	list the row labels and column names
ndim	number of dimensions
size	number of elements
shape	return a tuple representing the dimensionality
values	numpy representation of the data

Data Frames attributes

- Find how many records/instances/samples this data frame has; df.shape[0]
- How many elements are there? df.size
- What are the column names? df.columns
- What data types of columns we have in this data frame? df.dtypes

Data Frames methods/functions

- Unlike attributes, Python methods/functions have *parenthesis.*
- All attributes and methods can be listed with a *dir()* function: `dir(df)`

df.method()	description
head([n]), tail([n])	Print first/last n rows
describe()	generate descriptive statistics (for numeric columns only)
max(), min()	return max/min values for all numeric columns
mean(), median()	return mean/median values for all numeric columns
std()	standard deviation
sample([n])	returns a random sample of the data frame
dropna()	drop all the records with missing values

Data Frames methods/functions

- Give the summary for the numeric columns in the dataset: df.describe()
- Calculate standard deviation for all numeric columns; df.std()
- What are the mean values of the first 50 records in the dataset? ***Hint:*** use head() method to subset the first 50 records and then calculate the mean

  df.head(50).mean()

Selecting a column in a Data Frame

- *Method 1:* Subset the data frame using column name:

  df[‘Age']

- *Method 2*: Use the column name as an attribute:

  df.Age

Data Frames *groupby* method

- Using "group by" method we can:
  - Split the data into groups based on some criteria
  - Calculate statistics (or apply a function) to each group

In [ ]:
```
#Group data using rank
df_rank = df.groupby(['rank'])
```

In [ ]:
```
#Calculate mean value for each numeric column per each group
df_rank.mean()
```

rank	phd	service	salary
AssocProf	15.076923	11.307692	91786.230769
AsstProf	5.052632	2.210526	81362.789474
Prof	27.065217	21.413043	123624.804348

Data Frames *groupby* method

- Once groupby object is created, we can calculate various statistics for each group:

```
In [ ]: #Calculate mean salary for each professor rank:
        df.groupby('rank')[['salary']].mean()
```

rank	salary
AssocProf	91786.230769
AsstProf	81362.789474
Prof	123624.804348

Note: If single brackets are used to specify the column (e.g. salary), then the output is Pandas Series object. When double brackets are used the output is a Data Frame

Data Frame: filtering

To subset the data we can apply Boolean indexing. This indexing is commonly known as a filter. For example if we want to subset the rows in which the salary value is greater than $120K:

In [ ]:
```
# salary more than 120K:
df_sub = df[ df['salary'] > 120000 ]
```

Any Boolean operator can be used to subset the data:

> greater; >= greater or equal;

< less; <= less or equal;

== equal; != not equal;

In [ ]:
```
#Select only those rows that contain female professors:
df_f = df[ df['sex'] == 'Female' ]
```

Data Frames: Slicing

- There are a number of ways to subset the Data Frame:
  - one or more columns
  - one or more rows
  - a subset of rows and columns

- Rows and columns can be selected by their position or label

Data Frames: Selecting rows

If we need to select a range of rows, we can specify the range using ":"

```
In [ ]: #Select rows by their position:
        df[10:20]
```

Notice that the first row has a position 0, and the last value in the range is omitted:
So for 0:10 range the first 10 rows are returned with the positions starting with 0 and ending with 9

Data Frames: method loc

The loc() function is label based data selecting method which means that we have to pass the name of the row or column which we want to select. If we need to select a range of rows, using their labels we can use method loc:

In [ ]:
```
#Select rows by their labels:
df_sub = df[ df['rank'] == 'Prof' ]
df_sub.loc[10:20,['rank','sex','salary']]
```

Out[ ]:

	rank	sex	salary
10	Prof	Male	128250
11	Prof	Male	134778
13	Prof	Male	162200
14	Prof	Male	153750
15	Prof	Male	150480
19	Prof	Male	150500

Data Frames: method iloc

The iloc() function is an indexed-based selecting method which means that we have to pass an integer index in the method to select a specific row/column. This method does not include the last element of the range passed in it unlike loc(). If we need to select a range of rows and/or columns, using their positions we can use method iloc:

In [ ]:
```
#Select rows by their labels:
df_sub.iloc[25:50,[0, 3, 4, 5]]
```

Out[ ]:

	rank	service	sex	salary
26	Prof	19	Male	148750
27	Prof	43	Male	155865
29	Prof	20	Male	123683
31	Prof	21	Male	155750
35	Prof	23	Male	126933
36	Prof	45	Male	146856
39	Prof	18	Female	129000
40	Prof	36	Female	137000
44	Prof	19	Female	151768
45	Prof	25	Female	140096

Data Frames: method iloc (summary)

```
df.iloc[0]  # First row of a data frame
df.iloc[i]  #(i+1)th row
df.iloc[-1] # Last row
```

```
df.iloc[:, 0]  # First column
df.iloc[:, -1] # Last column
```

```
df.iloc[0:7]        #First 7 rows
df.iloc[:, 0:2]     #First 2 columns
df.iloc[1:3, 0:2]   #Second through fourth rows and first 2 columns
df.iloc[[0,5], [1,3]]  #1st and 6th rows and 2nd and 4th columns
```